

Concorrência

Exclusão mútua e sincronização

Sistemas Operativos

2024/2025

Baseado nos slides do **Capítulo 5 (Concurrency: Mutual Exclusion and Synchronization)** do livro Operating Systems: Internals and Design Principles, 7th Edition, Dave Bremer, Otago Polytechnic, N.Z., 2008, Prentice Hall.

Múltiplos Processos

Desenho de sistemas operativos preocupa-se com a gestão de processos e threads:

- Multiprogramming
- Multiprocessamento
- Processamento distribuído

Surge em 3 contextos diferentes

- **Múltiplas aplicações**

- em processamento multiprogramming, o tempo é partilhado de forma dinâmica com o número de aplicações ativas.

- **Aplicações estruturadas**

- algumas aplicações podem ser efetivamente programadas como um conjunto de processos concorrentes.

- **Sistemas operativos estruturados**

- sistemas operativos são normalmente implementados como um conjunto de processos ou threads.

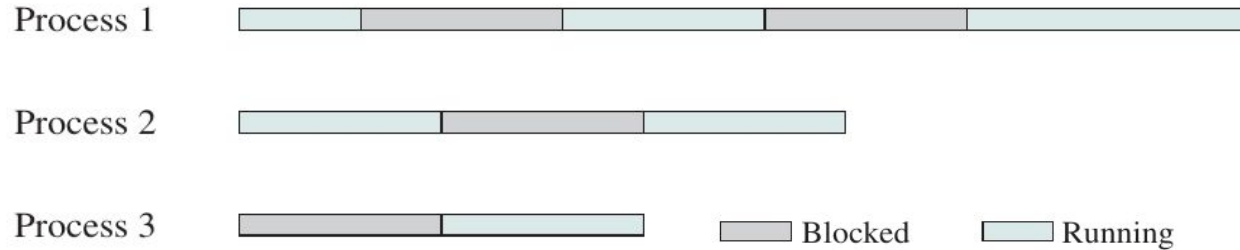
Key Terms: Concorrência

Table 5.1 Some Key Terms Related to Concurrency

atomic operation	A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

- **Interleaving e overlapping**

- (intercalar e sobrepor)
- podem ser encarados como exemplos de processamento concorrente
- ambos apresentam os mesmos problemas



Overlapping

Interleaving

- **Uniprocessador (aplicação de interleaving)**

- não é possível prever o tempo relativo de execução de um processo
- depende das atividades de outros processos
- da forma como o SO trata as interrupções
- das regras de escalonamento do SO

- **Multiprocessador (aplicação de interleaving + overlapping)**

- + processos em simultâneo -> fundamentalmente os mesmos problemas de Uniprocessador

- **Partilha** de recursos globais
 - A ordem das escritas e leituras ao mesmo recurso são críticas.
 - Ordens diferentes produzem resultados diferentes
- Dificuldade do SO em fazer a gestão da **alocação** dos recursos de forma ótima
 - Um processo A tem o controlo de um recurso de I/O quando é suspenso (suspended)
 - Se o SO simplesmente bloqueia esse recurso a outros processos podemos entrar num Deadlock
- Dificuldade em **encontrar erros de programação**
 - Os resultados não são determinísticos e reproduzíveis

Exemplo de Concorrência

2 processos P1 e P2 executam o procedimento partilhado echo()

- chin e chout são variáveis partilhadas

Uma possível execução:

Process P1

-
- `chin = getchar();`
-
- `chout = chin;`
- `putchar(chout);`
-
-

Process P2

-
-
- `chin = getchar();`
- `chout = chin;`
-
- `putchar(chout);`
-

```
void echo()  
{  
    chin = getchar();  
    chout = chin;  
    putchar(chout);  
}
```

Considerando que:

- A primeira execução do `getchar()` retorna A
- E a segunda B

O resultado desta execução é BB

- Ocorre quando vários processos ou threads lêem e escrevem dados partilhados
- Resultado final depende do timing relativo de execução
 - O “perdedor” da corrida é o processo que atualiza no final e que vai de determinar o valor final da variável

Exemplo

- P1 e P2 partilham a variável global *a*
- Primeiro, P1 atualiza *a* para o valor 1
- Depois, P2 atualiza *a* para o valor 2
- A variável *a* fica com o valor 2

Problemas de design e gestão criados pela existência de concorrência

- O sistema operativo deve:
 - ser capaz de manter o rasto dos vários processos
 - alocar e desalocar recursos para cada processo ativo
 - proteger dados recursos físicos de cada processo da interferência de outros processos
 - **garantir que processos e outputs são independentes da velocidade de processamento**

Interação de processos

Table 5.2 Process Interaction

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

Resource Competition

- Processos concorrentes entram em conflito quando estão a competir pelo uso do mesmo recurso
 - Por exemplo: dispositivos de I/O, memória, tempo de processador, relógio, etc...

Exclusão mútua

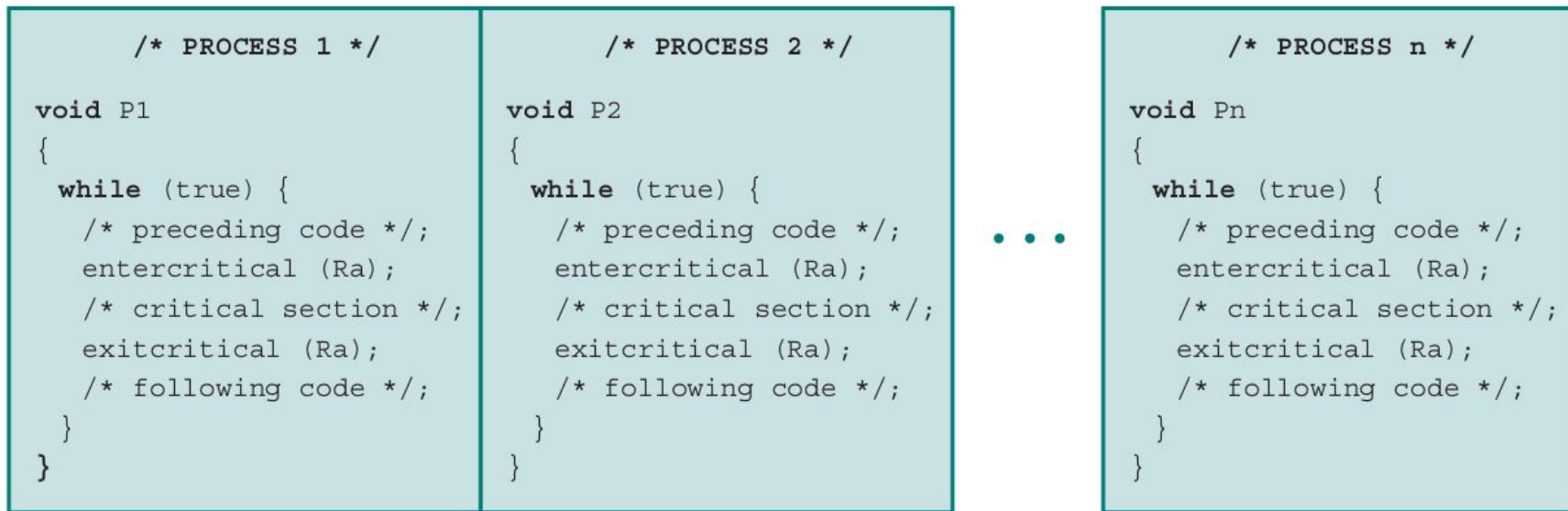


Figure 5.1 Illustration of Mutual Exclusion

Requisitos para exclusão mútua

- Deve ser garantida
- Um processo que termina, deve fazê-lo sem que haja interferência noutros processos
- Não pode existir deadlock ou starvation
- Não pode ser negado o acesso de um programa a uma secção crítica, quando já não existem outros processos a usarem essa secção.
- Não fazer suposições sobre a velocidade relativa ou número de processos
- Um processo só pode estar numa secção crítica durante um tempo limitado

Exclusão mútua: suporte no hardware

Desligar as interrupções

- Sistemas uniprocessador
- Garante a exclusão mútua

```
while (true) {  
    /* disable interrupts */;  
    /* critical section */;  
    /* enable interrupts */;  
    /* remainder */;  
}
```

Desvantagens

- A eficiência pode degradar de forma significativa
- Esta abordagem não funciona numa arquitetura multi processador

Instruções máquina especiais

- Instrução **Compare&Swap**

- também conhecida de “compre and exchange instruction”
- uma **comparação (compare)** é feita entre um valor da memória (*word) e um valor de teste (testval)
- se os valores forem os mesmos é feita uma **troca (swap)**
- executado de forma **atômica**

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval;
    oldval = *word
    if (oldval == testval) *word = newval;
    return oldval;
}
```

Instrução Compare&Swap

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */; ←
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ... ,P(n));
}
```

- Os processos ficam num estado de busy-waiting
 - Também conhecido como spin waiting
 - Nada de relevante acontece até bolt ter o valor 0

(a) Compare and swap instruction

Figure 5.2 Hardware Support for Mutual Exclusion

Instrução Exchange

- troca o conteúdo de um registo com o de uma localização em memória
- executado de forma **atômica**

```
void exchange (int *register, int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P(int i)
{
    int keyi = 1;
    while (true) {
        do exchange (&keyi, &bolt)
        while (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

(b) Exchange instruction

Figure 5.2 Hardware Support for Mutual Exclusion

Vantagens

- Aplicável a qualquer número de processos, em uni ou multi processador, partilhando a memória principal
- Simples e fácil de verificar
- Pode ser usada para suportar várias secções críticas
 - cada secção pode ser definida pela sua variável

Desvantagens

- Usa ***busy-waiting***
 - um processo está à espera de acesso a uma secção crítica enquanto está a consumir tempo de processador
- É possível existir ***starvation*** quando um processo saí de uma secção crítica e existe mais do que um processo há espera
- É possível a existência de ***deadlock***

Mecanismos de concorrência comuns

Table 5.3 Common Concurrency Mechanisms

Semaphore	An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a counting semaphore or a general semaphore .
Binary Semaphore	A semaphore that takes on only the values 0 and 1.
Mutex	Similar to a binary semaphore. A key difference between the two is that the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1).
Condition Variable	A data type that is used to block a process or thread until a particular condition is true.
Monitor	A programming language construct that encapsulates variables, access procedures, and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are <i>critical sections</i> . A monitor may have a queue of processes that are waiting to access it.
Event Flags	A memory word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR).
Mailboxes/Messages	A means for two processes to exchange information and that may be used for synchronization.
Spinlocks	Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability.

Semáforos

1. Podem ser **initialized** com um valor inteiro não negativo
2. A operação **semWait** decrementa o seu valor
3. A operação **semSignal** incrementa o seu valor

Ambas as operações semWait e semSignal são **atómicas**

- **Não existe forma de saber** antes do processo decrementar o semáforo, **se vai ficar bloqueado ou não.**
- Depois de um processo incrementar um semáforo e outro processo acordar, ambos os processos continuam a executar de forma concorrente. **Não existe forma de saber qual processo, ou se ambos, continuam imediatamente num sistema uniprocessador.**
- Quando se assinala um semáforo, **não se sabe se existe outro processo à espera .**

```
struct semaphore {
    int count;
    queueType queue;
};

void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

Figure 5.3 A Definition of Semaphore Primitives

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};

void semWaitB(binary_semaphore s)
{
    if (s.value == one)
        s.value = zero;
    else {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignalB(semaphore s)
{
    if (s.queue is empty())
        s.value = one;
    else {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

Figure 5.4 A Definition of Binary Semaphore Primitives

Semáforos Strong/Weak

- Uma fila é usada para guardar os processos em espera no semáforo
- FIFO policy
 - Semáforo Strong
- No policy
 - Semáforo Weak

Exclusão mútua

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), ..., P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

Problema Produtor consumidor

- O problema do produtor consumidor é um cenário clássico de sincronização em sistemas operativos e concorrência.
- Consiste em dois tipos de processos:
 - **Produtores** produzem dados
 - **Consumidores** consomem esses dados
- Condições que devem ser satisfeitas:
 - Produtores só produzem dados quando houver espaço disponível no buffer
 - Consumidores só consomem dados quando houver dados no buffer
 - O buffer só pode ser modificado por um processo de cada vez

Produtor consumidor usando semáforos

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n = 0, e = sizeofbuffer;
void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.13 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores

Implementação de semáforos

- Imperativo que o **semWait** e o **semSignal** possam ser implementados como primitivas atómicas
- Podem ser implementados em hardware ou firmware
- Existem esquemas de software que podem ser usados
 - e.g.: Algoritmo de Dekker ou Peterson
- Utilização de um esquema de exclusão mútua baseada em hardware

Alternativas a semáforos

- Monitors
- Rendez-vous
- Algoritmo de Dekker ou Petterson
- Baseado em hardware: swap, test&set

Exemplo - Readers/Writers Problem

- Uma área de dados é partilhada entre vários processos
 - Alguns processos apenas leem dados (readers)
 - Outros apenas escrevem (writers)
- Condições que devem ser satisfeitas:
 - qualquer número de leitores podem ler
 - apenas pode existir um escritor a escrever de cada vez
 - se um escritor estiver a escrever, nenhum leitor pode ler

Exemplo - Readers/Writers Problem

Leitores têm prioridade

- O semáforo wsem é utilizado para garantir a exclusão mútua.
- Apenas quando não existem leitores a tentar ler, wsem é libertado (signal)
 - A variável global readcount é utilizada para manter o registo do número de leitores
 - O semáforo x é utilizado para garantir que readcount é atualizado corretamente

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true){
        semWait (x);
        readcount++;
        if(readcount == 1)
            semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount;
        if(readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true){
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader,writer);
}
```

Figure 5.22 A Solution to the Readers/Writers Problem Using Semaphore: Readers Have Priority

Exemplo - Readers/Writers Problem

Escritores têm prioridade

Para evitar starvation de escritores adiciona:

- Um semáforo rsem que evita que os leitores se registem quando um escritor está à espera
- Uma variável writecount que controla o registo em rsem
- Um semáforo y que controla a atualização de writecount

```
/* program readersandwriters */  
int readcount, writecount;  
void reader()  
{  
    while (true){  
        semWait (z);  
        semWait (rsem);  
        semWait (x);
```

...

```
void writer ()  
{  
    while (true){  
        semWait (y);  
        writecount++;  
        if (writecount == 1)  
            semWait (rsem);  
        semSignal (y);  
        semWait (wsem);  
        WRITEUNIT();  
        semSignal (wsem);  
        semWait (y);  
        writecount;  
        if (writecount == 0) semSignal (rsem);  
        semSignal (y);  
    }  
}  
void main()  
{  
    readcount = writecount = 0;  
    parbegin (reader, writer);  
}
```

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority

Exemplo - Readers/Writers Problem

Escritores têm prioridade

Para evitar starvation de escritores adiciona:

- Um semáforo rsem que evita que os leitores se registem quando um escritor está à espera
- Uma variável writecount que controla o registo em rsem
- Um semáforo y que controla a atualização de writecount

```
/* program readersandwriters */
int readcount, writecount;
void reader()
{
    while (true){
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}

void writer ()
{
    while (true){
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}

void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

Figure 5.23 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority

Estados das filas no Exemplo - Readers/Writers

- Para os leitores, é necessário um semáforo adicional
 - Apenas um leitor é permitido na fila em *rsem*
 - Caso contrário voltamos a ter starvation nos escritores
 - Leitores adicionais ficam na fila do semáforo *z*, imediatamente antes de esperarem em *rsem*

Table 5.6 State of the Process Queues for Program of Figure 5.23

Readers only in the system	• <i>wsem</i> set • no queues
Writers only in the system	• <i>wsem</i> and <i>rsem</i> set • writers queue on <i>wsem</i>
Both readers and writers with read first	• <i>wsem</i> set by reader • <i>rsem</i> set by writer • all writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>
Both readers and writers with write first	• <i>wsem</i> set by writer • <i>rsem</i> set by writer • writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>